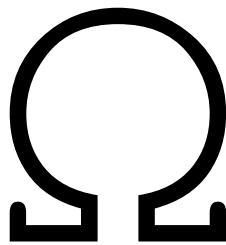


Backed Atomic Swap

Final Audit Report

November 28, 2025



Team Omega

`Teamomega.eth.limo`

Summary	2
Scope of the Audit	3
Resolution	3
Methods Used	3
Disclaimer	4
Severity definitions	4
Findings	5
General	5
G1. Pin solidity dependencies in package.json [low] [resolved]	5
G2. No continuous integration [info] [not resolved]	5
G3. Documentation is outdated [info] [resolved]	5
AtomicSwapUpgradeable.sol	6
A1. The signer can take all tokens that are approved to the swap contract [medium] [resolved]	6
A2. QuoteInvalidated event is emitted also for valid quotes [low] [resolved]	7
A3. The SwapExecuted omits the addresses the tokens as sent to [low] [resolved]	7
A4. invalidateQuote permissions could be set to signer [info] [resolved]	7
A5. The Permit struct is inconsistently implemented [info] [not resolved]	7

Summary

Backed Finance has asked Team Omega to audit the contracts that define the behavior of contract for atomic swaps.

We found **no high severity issue** - these are issues that can lead to a loss of funds, and are essential to fix. We classified **1** issue as “medium” - these are issues we believe you should definitely address. We did classify **3** issues as “low”, and an additional **4** issues were classified as “info” - we believe the code would improve if these issues were addressed as well.

All significant issues were subsequently resolved by the Backed team.

Severity	Number of issues	Number of resolved issues
High	0	0
Medium	1	1
Low	3	3
Info	4	2

Scope of the Audit

Scope of the Audit

The audit concerns an upgrade of the Backed token contracts and the CCIP bridge.

The audit concerns a contract for swapping tokens

.

As of writing, the code is still under development in the following repository:

```
https://github.com/backed-fi/swap-contract
```

We audited the code at the following commit:

```
c7f253c9e86e1b3f12c38c90c116a15bce4f6d71
```

Resolution

After delivering a first report on October 14, 2025, all significant issues were addressed in the following commit:

```
2eb1b6c1b1b2e6a123d5defb997533b90a54e4bd
```

We reviewed the changes and updated the issues below accordingly.

Methods Used

The contracts were compiled, deployed, and tested in a test environment.

Code Review

We manually inspected the source code to identify potential security flaws. We also inspected the context in which the contracts were developed, namely the Chainlink Bridge architecture.

Automatic analysis

We have used static analysis tools to detect common potential vulnerabilities. No high severity issues were identified with the automated processes. Some low severity issues were found, and we have included them below in the appropriate parts of the report.

Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Severity definitions

High	Vulnerabilities that can lead to loss of assets or data manipulations.
Medium	Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations
Low	Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc
Info	Matters of opinion

Findings

General

G1. Pin solidity dependencies in package.json [low] [resolved]

It is desirable to have a deterministic outcome when compiling smart contracts. This makes the output of the compiler more predictable, which helps when verifying contract deployments, and reduces the chance of introducing new vulnerabilities when rebuilding the package.

Currently, `package.json` specifies a range for the OpenZeppelin dependency:

```
"dependencies": {
  "@openzeppelin/contracts": "^5.4.0",
  "@openzeppelin/contracts-upgradeable": "^5.4.0"
}
```

Recommendation: Pin a specific version of the dependency:

```
"@openzeppelin/contracts": "5.4.0",
"@openzeppelin/contracts-upgradeable": "5.4.0",
```

Severity: Low

Resolution: The issue was resolved as recommended

G2. No continuous integration [info] [not resolved]

The github repository does not have continuous integration actions defined, which makes it harder to guarantee that the code will compile, and the tests will run.

Recommendation: Configure continuous integration and run the tests in a github action.

Severity: Info

Resolution: The issue was not resolved

G3. Documentation is outdated [info] [resolved]

The documentation is outdated, in particular the file `ATOMIC_SWAP_CONTRACT_SPEC`

Recommendation: Fix the documentation.

Severity: Info

Resolution: Some outdated files were removed

AtomicSwapUpgradeable.sol

A1. The signer can take all tokens that are approved to the swap contract [medium] [resolved]

In the current configuration, the signer can execute any order they want, provided it is less than the amount that the `swapMessage.incomingTransfer.from` address approves or permits.

There are two ways that a trading account can grant permission to the Swap contract.

The trading account can either approve the swap contract to spend tokens directly (by calling `approve` on the `incomingTransfer.token`). The `signer` can now sign any `SwapMessage` – for example, a message that sends the approved tokens to an address of their choice and sends no tokens in return – call `executeSwap` with a permit that has `address(0)` as the owner (which will skip checking for the signature).

The second way a trading account can give the contract permission to spend tokens is by signing a EIP-2612 `Permit` object. In a typical flow, the trading account will then call `executeSwap` with a `SwapMessage` of the quote they want to execute and the signed `Permit` to send the tokens. The `signer` can frontrun this transaction, taking the signed `Permit` but replacing the `SwapMessage` with a swap that has completely different conditions.

The basic trust model here is broken: at the surface it looks as if the signer of the `Permit` gives the `SwapContract` an allowance to spend their tokens under very specific conditions, while actually, the `signer` can spend the tokens in any way they want.

Note that if the communication channel in which the `signer` sends signed `SwapMessages` to the trading account is not secure, an attacker could use the same frontrunning technique and replace any `SwapMessage` with another message they have intercepted (provided it is signed by the `signer` and not expired).

Recommendation: An easy way to avoid this is to require that `msg.sender == swapMessage.incomingTransfer.from`. This limits the functionality slightly, in that the trading account must submit the transactions themselves. Another approach is to require that `swapMessage.incomingTransfer.from` signs not only the permit, but also the `SwapMessage`, and add this signed message in the `executeSwap` parameters.

Resolution: The issue was resolved as recommended in our first suggestion

A2. QuoteInvalidated event is emitted also for valid quotes [low] [resolved]

If `invalidateQuote` is called with a `quoteId` of a quote that was already used, the `QuoteInvalidated` is emitted. This may be misleading, and could cause issues when showing such data on the front end.

Recommendation: Only emit the `QuoteInvalidated` event if the quote was actually invalidated in this call.

Resolution: The issue was resolved as recommended

A3. The SwapExecuted omits the addresses the tokens as sent to [low] [resolved]

The `SwapExecuted` event does not include the address for `message.incomingTransfer.to` and `message.outgoingTransfer.to`.

Recommendation: Include these addresses in the Event.

Resolution: The issue was resolved as recommended

A4. invalidateQuote permissions could be set to signer [info] [resolved]

Currently, it is the `signer` of the contract that creates quotes, and the `owner` of the contract can invalidate these quotes by calling `invalidateQuote`.

It seems more consistent that it is the same agent that creates and invalidates quotes.

Recommendation: Allow also for the signer of the contract to call `invalidateQuote`, or define a new role for it.

Resolution: The issue was resolved as recommended

A5. The Permit struct is inconsistently implemented [info] [not resolved]

The EIP-2612 defines a precise way to sign permissions: the owner of the token signs a struct specifying `owner`, `spender`, `value`, `nonce` and `deadline`. In the current code, the values `v`, `r` and `s` encode an ERC-2612 Permit object with these values.

The `Permit` struct as defined in `AtomicSwapUpgradeable` omits the values for `spender` and `value`, which makes sense as these values are, respectively, given the the address of the Swap contract, and the `incomingTransfer.amount` of token.

To be consistent, however, you can then also omit the value for `owner` (as that is given by the `incomingTransfer.from` address) and, if you want to be very economical, you could use the expiration of the `SwapMessage` as the value for the deadline.

Recommendation: Use `incomingTransfer.from` instead of passing `Permit.owner`.